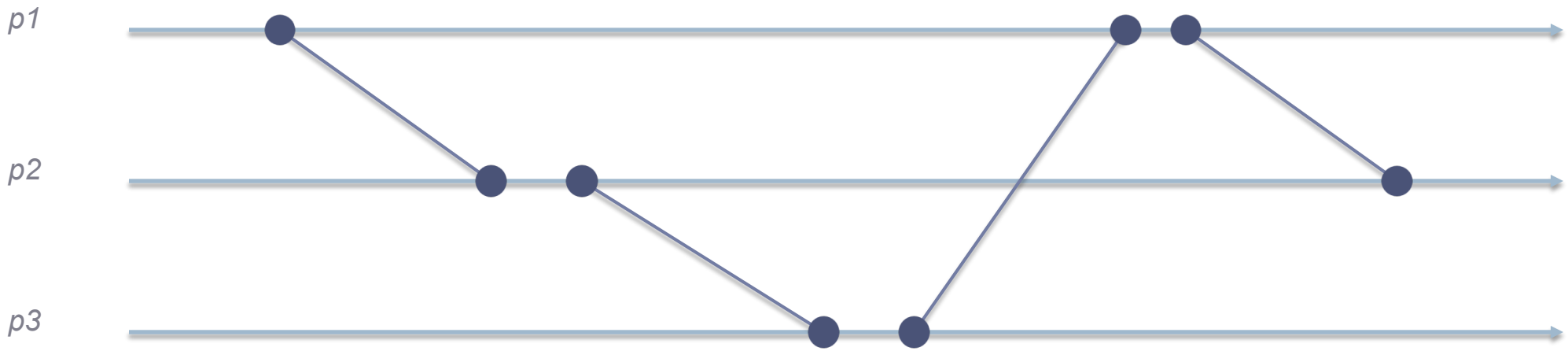
Redes de Comunicaciones y Cómputo Distribuido



2do. Cuatrimestre de 2026

Licenciatura en Ciencias de la Computación
Departamento de Computación
Facultad de Cs. Exactas y Naturales
Universidad de Buenos Aires

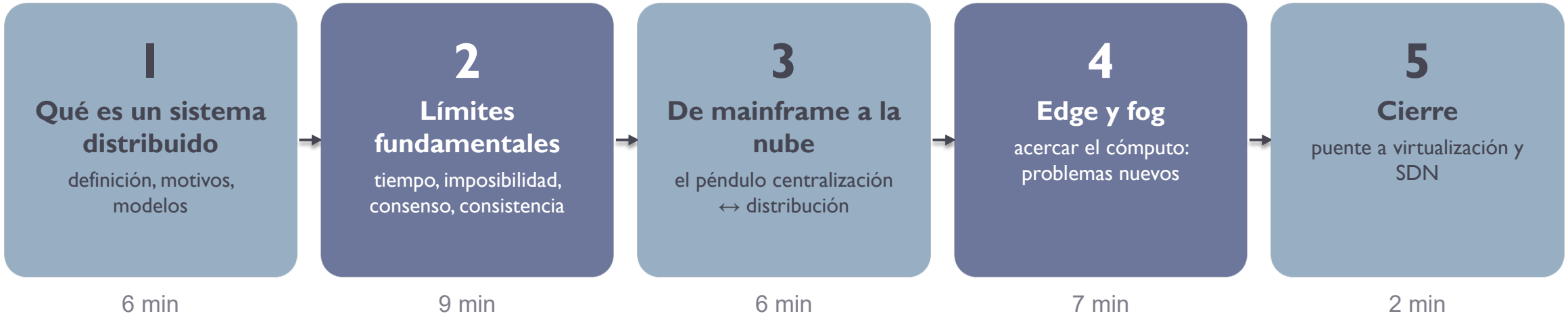




Cómputo distribuido: de la nube al edge

Modelos, límites y arquitecturas — introducción a cloud, edge y fog

Hilo conductor (30 minutos)



Tesis de la clase

Cloud, edge y fog no son productos sino respuestas distintas a las mismas preguntas del cómputo distribuido: dónde está el estado, cómo se replica, qué se sacrifica ante una partición y cuánto cuesta coordinar. Las restricciones no cambian; cambia dónde se ubica el cómputo respecto de los datos y de los usuarios.

¿Qué es un sistema distribuido?

"Un sistema distribuido es aquel en el que la falla de una computadora que ni sabías que existía puede dejar inutilizable tu propia computadora."

Leslie Lamport, 1987

"Una colección de computadoras independientes que se presenta a sus usuarios como un único sistema coherente."

Tanenbaum y van Steen

Tres rasgos que lo definen (Coulouris et al.)

Concurrencia	Sin reloj global	Fallas independientes
los componentes ejecutan en paralelo y comparten recursos; aparecen las carreras y la necesidad de coordinar	cada nodo tiene su propio reloj, que deriva; no hay una noción única de "ahora"	un nodo o un enlace puede fallar mientras el resto sigue; a veces nadie sabe que falló

La comunicación es sólo por mensajes: no hay memoria compartida. Todo lo que un nodo sabe de otro es lo que le llegó, y puede estar desactualizado.

¿Por qué distribuir? Y sus límites

1. **Escalar**
   más usuarios o datos de los que entran en una máquina

2. **Tolerar fallas**
   redundancia: que el servicio sobreviva a la caída de un nodo

3. **Localidad**
   poner el cómputo cerca de los usuarios o de los datos

4. **Costo**
   muchas máquinas comunes en lugar de una excepcional

Escalar verticalmente (una máquina más grande) tiene techo físico y económico; **escalar horizontalmente** (más máquinas) no, pero exige coordinar.

Ley de Amdahl (1967)

$$S(n) = 1 / ((1 - p) + p / n)$$

p: fracción paralelizable; n: número de nodos. Con $n \to \infty$, $S \to 1 / (1 - p)$.

p = 0,95	n = 8	n = 64	n = 1024
Aceleración S(n)	5,9	15,4	19,6

Un 5 % secuencial (coordinación, consenso, un lock) limita la aceleración a 20×, con cualquier número de máquinas. Gustafson (1988): si el problema crece con n, el panorama mejora.

Modelos de sistema: qué suponemos

Tiempo	Fallas de procesos	Canales
Síncrono cotas conocidas para cómputo y mensajes	**Crash** el proceso se detiene y no vuelve	**Confiables** todo mensaje enviado llega, una vez
Parcialmente síncrono las cotas existen pero no se conocen, o valen desde algún momento (DLS, 1988)	**Crash-recovery** se detiene y vuelve, con su estado persistente	**Con pérdidas equitativas** pueden perder, pero si se reintenta, llega
Asíncrono sin cotas: un mensaje puede tardar arbitrariamente	**Bizantina** comportamiento arbitrario, incluso malicioso	**Arbitrarios** pérdida, duplicación, reordenamiento

Regla de oro: todo algoritmo distribuido es correcto respecto de un modelo. La pregunta no es si funciona, sino bajo qué supuestos de tiempo, fallas y canales.

Las ocho falacias de la computación distribuida

1
La red es confiable

2
La latencia es cero

3
El ancho de banda es infinito

4
La red es segura

5
La topología no cambia

6
Hay un solo administrador

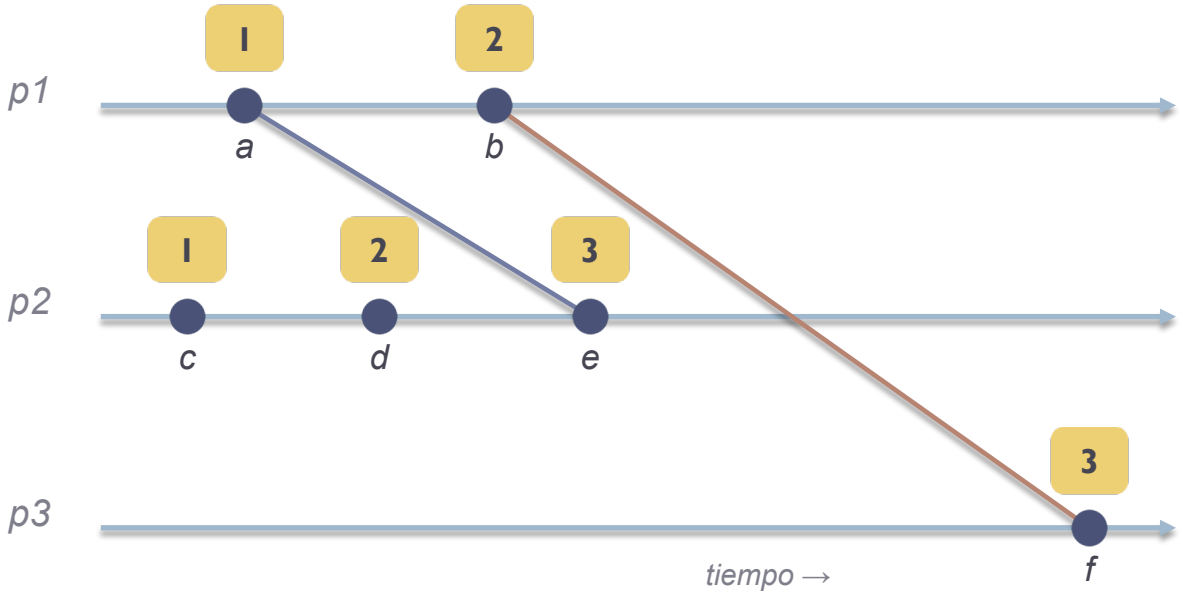
7
El costo de transporte es cero

8
La red es homogénea

Deutsch (Sun, 1994) y Gosling (1997)

Cada falacia es una hipótesis que los programadores hacen sin darse cuenta. Las tres primeras son las que más cuestan: una llamada remota no es una llamada local. En el edge, las ocho se vuelven más falsas todavía.

Relojes de Lamport: ordenar sin reloj común



e recibe m1 con t = 1:
max(2, 1) + 1 = 3
gana el reloj local

f recibe m2 con t = 2:
max(0, 2) + 1 = 3
gana la marca del mensaje

"Ocurrió antes" (a → b)

a y b en el mismo proceso, a primero; o a es el envío y b la recepción de un mismo mensaje; o por transitividad.

Tres reglas: un contador entero C por proceso

1. Evento local o envío: C := C + 1
2. Al enviar: el mensaje lleva t = C
3. Al recibir m con marca t: C := max(C, t) + 1

El max asegura que la recepción quede después del envío, aunque el receptor venía atrasado.

Garantía: a → b ⇒ C(a) < C(b). **La recíproca no vale:** C(c) = 1 < C(b) = 2, pero c y b son concurrentes: ninguno pudo influir en el otro.

Más allá de Lamport: vectores y relojes físicos

El mismo diagrama, con relojes vectoriales

Evento	Lamport	Vectorial (p1, p2, p3)
a	1	(1, 0, 0)
b	2	(2, 0, 0)
c	1	(0, 1, 0)
d	2	(0, 2, 0)
e	3	(1, 3, 0)
f	3	(2, 0, 1)

Cada proceso cuenta los eventos de todos; al recibir, máximo componente a componente y suma 1 en la propia.

c = (0,1,0) y b = (2,0,0) no se pueden comparar ⇒ concurrentes. Costo: n enteros por mensaje.

Relojes físicos: derivan

Un cristal de 20 ppm se desvía $20 \cdot 10^{-6} \times 86\,400\ \text{s} \approx 1{,}7\ \text{s}$ por día.

NTP los acerca a milisegundos en Internet; PTP, a microsegundos en una LAN. Nunca a cero.

TrueTime (Spanner, Google, 2012)

El reloj no devuelve un instante sino un intervalo $[t - \varepsilon, t + \varepsilon]$, con ε de pocos milisegundos gracias a GPS y relojes atómicos.

Antes de confirmar una transacción, se espera ε: la incertidumbre del reloj se paga en latencia.

Lo que no se puede: resultados de imposibilidad

Dos Generales

1975 · Akkoyunlu et al.; Gray, 1978

Con un canal que puede perder mensajes, ningún protocolo garantiza que dos procesos acuerden actuar juntos. Siempre falta "el último ACK".

FLP

1985 · Fischer, Lynch y Paterson

En un sistema asíncrono, ningún algoritmo determinista de consenso termina siempre si puede fallar un solo proceso por crash.

CAP

2000 · Brewer; prueba: Gilbert y Lynch, 2002

Ante una partición de red, un sistema replicado debe elegir entre consistencia (linealizabilidad) y disponibilidad.

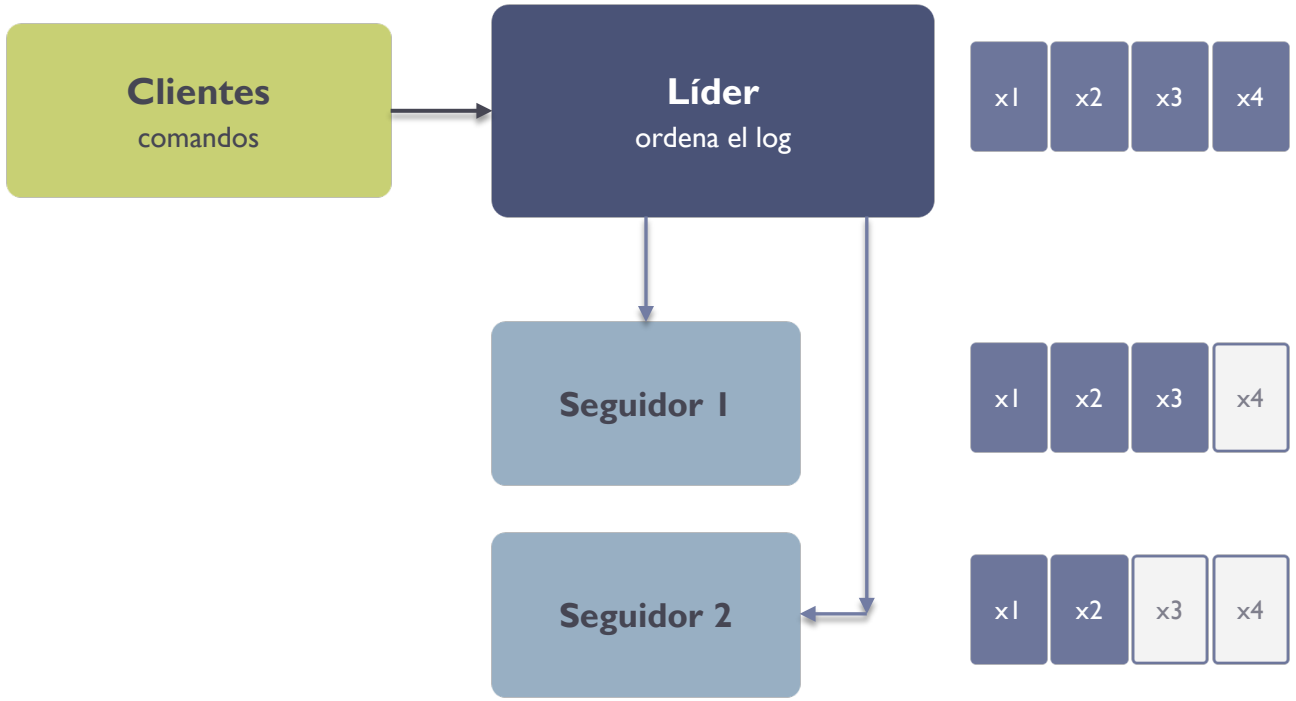
PACELC

2012 · Abadi

Si hay Partición: A o C. En otro caso (Else), igual se elige entre Latencia y Consistencia. La partición es rara; la latencia, permanente.

Cómo se vive con esto: aleatoriedad, timeouts (sincronía parcial), detectores de falla, y elegir explícitamente qué sacrificar.

Replicar estado: máquinas de estado y quórums



Máquina de estados replicada (Lamport, 1978; Schneider, 1990): si todas las réplicas aplican los mismos comandos en el mismo orden, terminan en el mismo estado. El problema se reduce a acordar el orden del log: consenso.

Cuántas réplicas hacen falta

- **Crash:** $n \geq 2f + 1$ (Paxos, Raft)
- **Bizantina:** $n \geq 3f + 1$ (PBFT)

Dos mayorías cualesquiera se intersectan: siempre hay un testigo de la última decisión.

Quórums de lectura y escritura

Con N réplicas, escribir en W y leer de R:

$R + W > N \Rightarrow$ toda lectura ve la última escritura

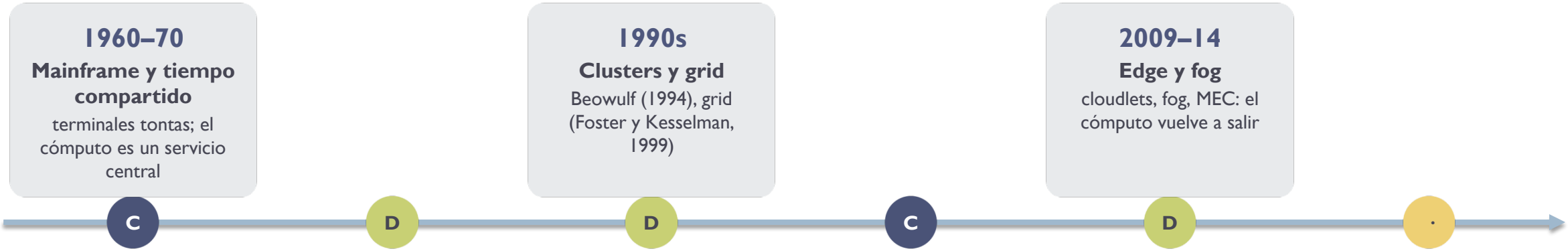
N = 3, W = 2, R = 2: tolera una caída. Bajar R o W compra latencia a cambio de consistencia (PACELC).

Modelos de consistencia: un espectro de costos

		requiere	ejemplos
Linealizable	cada operación parece instantánea en un punto entre su inicio y su fin	consenso en cada escritura	*Spanner, etcd*
Secuencial	un orden total común, compatible con el de cada proceso	orden total	*memoria multiprocesador idealizada*
Causal	si a pudo influir en b, todos ven a antes que b	metadatos causales	*COPS, chats, redes sociales*
Eventual	si cesan las escrituras, las réplicas convergen	ninguna coordinación síncrona	*DNS, Dynamo, CRDTs*

Hacia abajo: menos coordinación, menos latencia, más disponibilidad ante particiones, y más trabajo para el programador (conflictos, reconciliación).

De mainframe a la nube: el péndulo



C centralizar

D distribuir

Qué mueve el péndulo

La relación entre el costo de computar, el costo de comunicar y la latencia tolerable. Cuando la red es barata y rápida, se centraliza; cuando los datos o la latencia pesan más, se distribuye.

La nube vista como sistema distribuido

Las fallas son la norma

10 000 servidores, MTBF de 30 años cada uno:

10 000 / (30 × 365) ≈ 0,9 fallas por día

Replicar, reintentar y detectar fallas no es opcional.

La latencia de cola domina

Fan-out a 100 servidores, cada uno lento el 1 % de las veces:

$1 - 0{,}99^{100} \approx 63\ \%$

El p99 de cada nodo es la mediana del sistema.

Llevar el cómputo a los datos

MapReduce (2004): el programador escribe map y reduce; el sistema reparte, reintenta tareas fallidas y lanza copias de las lentas.

Pieza de la nube	Problema distribuido que resuelve
Almacén de configuración (etcd, Chubby)	consenso y elección de líder
Bases globales (Spanner) / clave-valor (Dynamo)	linealizabilidad vs disponibilidad

"El datacenter es la computadora"

Barroso y Hölzle: miles de máquinas programadas como una sola.

Edge y fog: cuando la nube queda lejos

La velocidad de la luz es un límite del diseño

En fibra, ≈ 200 000 km/s: 1 ms de ida y vuelta ≈ 100 km. Desde Buenos Aires, São Paulo está a ≥ 17 ms y Virginia a ≥ 84 ms.

Latencia	control, realidad aumentada y vehículos necesitan 1–20 ms
Volumen	1 000 cámaras × 4 Mb/s = 4 Gb/s que no conviene subir
Autonomía	seguir operando si se corta el enlace a la nube
Soberanía	datos que por ley o privacidad no salen del lugar



Fog: la jerarquía completa y la cooperación entre niveles (Cisco, 2012; IEEE 1934-2018). Edge: el borde de la red de acceso. Cloudlet: un "datacenter en una caja" cerca del usuario (2009).

El continuo: problemas distribuidos nuevos

Ubicación (placement)

Asignar tareas a nodos minimizando latencia con capacidad acotada: problema de asignación generalizada, NP-difícil. Heurísticas y aprendizaje.

Migración con movilidad

El usuario cambia de celda y su estado debe seguirlo. Migrar en vivo es copiar memoria que cambia: converge si la tasa de escritura es menor que la de copia.

Consistencia entre niveles

Fuerte implica latencia de la nube; eventual implica conflictos. PACELC en estado puro: el edge opera abajo en el espectro.

Autonomía ante particiones

El edge sigue aislado y reconcilia al volver. Los CRDT garantizan convergencia sin coordinar; los relojes híbridos ordenan sin sincronía perfecta.

Más nodos, más heterogéneos, más lejos y con enlaces peores: los límites del bloque 2 aprietan más en el edge que en la nube.

Para llevarse, y lo que viene

1. **Un sistema distribuido es conocimiento local y con retraso**
   concurrencia, sin reloj global, fallas independientes
2. **Hay límites que ningún diseño evita**
   Dos Generales, FLP, CAP/PACELC: elegir qué sacrificar es parte del diseño
3. **Replicar es acordar un orden**
   consenso con mayorías; quórums $R + W > N$; un espectro de consistencia
4. **Cloud, edge y fog ubican el cómputo**
   el péndulo se mueve según el costo de comunicar frente al de computar

Próxima clase: virtualización y SDN

Cómo la nube virtualiza cómputo y red, cómo se construye la red del datacenter, y por qué un controlador SDN es un sistema replicado con consenso.

Referencias

[1] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System”, CACM 21(7), 1978.

[2] M. Fischer, N. Lynch, M. Paterson, “Impossibility of Distributed Consensus with One Faulty Process”, JACM 32(2), 1985.

[3] C. Dwork, N. Lynch, L. Stockmeyer, “Consensus in the Presence of Partial Synchrony”, JACM 35(2), 1988.

[4] S. Gilbert, N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services”, SIGACT News, 2002.

[5] D. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design”, IEEE Computer 45(2), 2012.

[6] F. Schneider, “Implementing Fault-Tolerant Services Using the State Machine Approach”, ACM Computing Surveys, 1990.

[7] D. Ongaro, J. Ousterhout, “In Search of an Understandable Consensus Algorithm (Raft)”, USENIX ATC 2014.

[8] J. Corbett et al., “Spanner: Google’s Globally-Distributed Database”, OSDI 2012.

[9] G. DeCandia et al., “Dynamo: Amazon’s Highly Available Key-value Store”, SOSP 2007.

[10] J. Dean, L. A. Barroso, “The Tail at Scale”, CACM 56(2), 2013; J. Dean, S. Ghemawat, “MapReduce”, OSDI 2004.

[11] M. Satyanarayanan et al., “The Case for VM-Based Cloudlets in Mobile Computing”, 2009; F. Bonomi et al., “Fog Computing and Its Role in the IoT”, 2012.

[12] M. van Steen, A. S. Tanenbaum, Distributed Systems, 4a ed., 2023; G. Coulouris et al., Distributed Systems: Concepts and Design, 5a ed., 2011.